



SCHOOL OF COMPUTATION,
INFORMATION AND TECHNOLOGY —
INFORMATICS

TECHNISCHE UNIVERSITÄT MÜNCHEN

Bachelor's Thesis in Informatics

**Bidirectional Morphological Inflection
Generation and Analysis**

Murilo Escher Pagotto Ronchi





SCHOOL OF COMPUTATION,
INFORMATION AND TECHNOLOGY —
INFORMATICS

TECHNISCHE UNIVERSITÄT MÜNCHEN

Bachelor's Thesis in Informatics

**Bidirectional Morphological Inflection
Generation and Analysis**

**Bidirektionale morphologische
Flexionsgenerierung und -analyse**

Author:	Murilo Escher Pagotto Ronchi
Examiner:	Shu Okabe Ph.D.
Supervisor:	Advisor
Submission Date:	February 27, 2026



I confirm that this bachelor's thesis is my own work and I have documented all sources and material used.

Munich, February 27, 2026

Murilo Escher Pagotto Ronchi

Acknowledgments

Abstract

Contents

Acknowledgments	iv
Abstract	v
1 Introduction	1
2 Background	3
2.1 Tasks	3
2.1.1 Morphological Inflection	3
2.1.2 Morphological Analysis	4
2.2 Inflection Baselines	5
2.2.1 Non-Neural Baseline	5
2.2.2 Neural Baseline	7
2.3 ByT5	7
2.4 Large Language Models	8
3 Datasets and Languages	9
3.1 Datasets	9
3.1.1 UniMorph	9
3.1.2 Universal Dependencies	10
3.1.3 Data Preparation	11
3.2 Languages	13
4 Methodology	15
4.1 Morphological Inflection	15
4.1.1 Non-Neural Baseline	15
4.1.2 Neural Baseline	15
4.1.3 ByT5	16
4.1.4 Evaluation	17
4.2 Morphological Analysis	18
4.2.1 ByT5	18
4.2.2 Large Language Models	18
4.2.3 Evaluation	19

Contents

4.3	Bidirectional Inflection and Analysis	19
4.4	Context-Aware Morphological Analysis	21
5	Results	22
5.1	Morphological Inflection	22
5.2	Morphological Analysis	22
6	Conclusion	25
	Abbreviations	26
	List of Figures	27
	List of Tables	28
	Bibliography	30

1 Introduction

Citation test (Lamport, 1994).

Acronyms must be added in `main.tex` and are referenced using macros. The first occurrence is automatically replaced with the long version of the acronym, while all subsequent usages use the abbreviation.

E.g. `\ac{TUM}`, `\ac{TUM}` $\Rightarrow$ Technical University of Munich (TUM), TUM

For more details, see the documentation of the `acronym` package¹.

See Table 1.1, Figure 1.1, Figure 1.2, Figure 1.3.

Table 1.1: An example for a simple table.

A	B	C	D
1	2	1	2
2	3	2	3

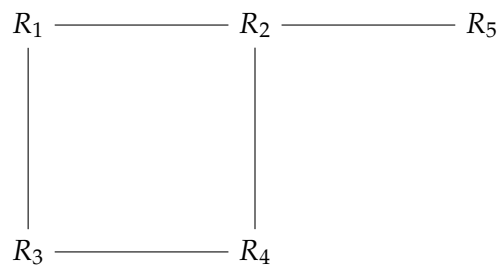


Figure 1.1: An example for a simple drawing.

¹<https://ctan.org/pkg/acronym>

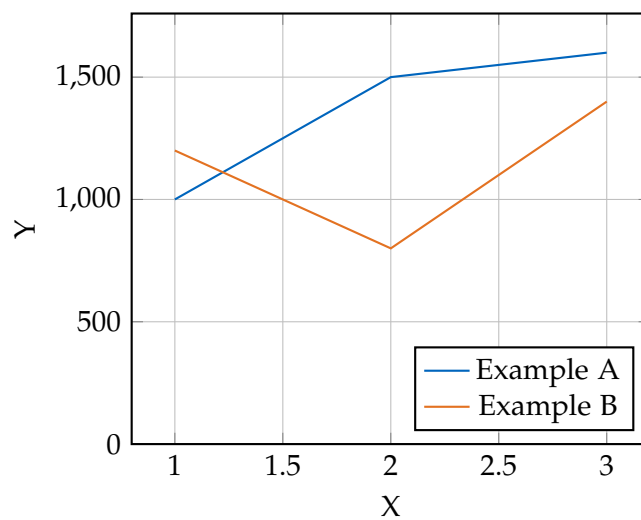


Figure 1.2: An example for a simple plot.

```
SELECT * FROM tbl WHERE tbl.str = "str"
```

Figure 1.3: An example for a source code listing.

2 Background

Different models have been used in this thesis to tackle two different, but related tasks. This chapter presents both tasks, the theoretical background and functionality behind the models used, as well as explanations of linguistic concepts pertaining to the tasks.

2.1 Tasks

This thesis addresses two closely related problems that can be viewed as inverse linguistic operations. Both tasks, introduced in the following subsections, involve applying a set of linguistic transformations to a base form of a word in order to produce an inflected form.

A lemma, commonly referred to as the canonical form, denotes the base representation of a word. This base form can undergo various transformations belonging to different grammatical categories, including mood, aspect, tense, case, number, and person. These transformations are represented as a collection of morphological features, which in this work are often referred to as *morphological tags*. Applying a specific set of such features to a lemma yields the corresponding inflected form.

2.1.1 Morphological Inflection

Morphological inflection refers to the process of generating a surface word form from a canonical base form (lemma) and a set of grammatical properties. Given a lemma and features describing tense, number, person, case, or other morphosyntactic categories, the task consists of producing the corresponding inflected form. An example in English is given in Figure 2.1.

$$(walk, \{\text{VERB, PAST, THIRD PERSON, SINGULAR}\}) \rightarrow walked$$

Figure 2.1: Illustration of the morphological inflection task. Given a lemma and a set of grammatical features, the goal is to generate the corresponding inflected form.

Automatic morphological inflection has long been studied in computational linguistics. Early approaches relied on rule-based systems and finite-state transducers (Koskenniemi, 1983; Kaplan and Kay, 1994; Beesley and Karttunen, 2003), which required curated rules determined by linguistic experts. More recently, the problem has been formulated as a supervised sequence-to-sequence learning task, in which neural encoder-decoder models operate directly on character sequences (Faruqui et al., 2016; Kann and Schütze, 2016; Aharoni and Goldberg, 2017). Such models learn morphological transformations from data and have become the dominant paradigm for the task.

The task has gained particular prominence through a series of multilingual shared tasks, most notably the SIGMORPHON and CoNLL–SIGMORPHON reinflection challenges (Cotterell et al., 2017; Cotterell et al., 2018; Goldman et al., 2023). These benchmarks provide standardized training and evaluation settings across dozens of typologically diverse languages and have established morphological inflection as a common testbed for studying character-level generalization and low-resource learning. Large-scale resources such as UniMorph further facilitate this research by offering broad cross-linguistic coverage and consistent morphological annotations (Batsuren et al., 2022).

2.1.2 Morphological Analysis

Full morphological analysis refers to the task of recovering both the lemma and the associated set of morphological descriptors from an inflected surface form. The task is closely related to the subtasks of lemmatization and morphological tagging, which predict the canonical form and grammatical features, respectively. An example in English is shown in Figure 2.2.

$$walked \rightarrow (walk, \{\text{VERB, PAST, THIRD PERSON, SINGULAR}\})$$

Figure 2.2: Illustration of the morphological analysis task. Given an inflected form, the goal is to predict the corresponding lemma and set of grammatical features.

Automatic morphological analysis has traditionally been addressed using rule-based and finite-state techniques similar to those employed for inflection. Classical two-level and finite-state morphology frameworks model analysis and generation within a single bidirectional system, enabling both the decomposition of surface forms into lexical representations and the production of inflected forms from abstract specifications (Koskenniemi, 1983; Beesley and Karttunen, 2003).

More recently, data-driven approaches have largely replaced manually engineered analyzers. Related problems are often formulated as lemmatization or morphological tagging, where models predict either the lemma or the set of grammatical features directly from the surface form. Neural character-level architectures have proven effective for these tasks, learning morphological regularities directly from data without explicit rules (Malaviya et al., 2019; Cotterell and Heigold, 2017). In this thesis, we adopt the more general formulation of full morphological analysis, which jointly predicts both the lemma and the complete set of morphological descriptors.

Beyond these task formulations, accurate morphological modeling plays an important role in downstream natural language processing applications more broadly. Generating and analyzing correct surface forms is essential for systems that must produce grammatically well-formed text, such as machine translation and text generation, and for tasks that rely on reliable morphological features, including tagging and syntactic parsing. Prior work has shown that incorporating morphological or subword structure improves neural machine translation for morphologically rich languages (Sennrich et al., 2016), while morphology-aware representations and feature prediction benefit language modeling and tagging performance (Botha and Blunsom, 2014; Cotterell and Heigold, 2017). These findings motivate the study of both inflection and analysis as complementary components of robust morphological processing.

2.2 Inflection Baselines

2.2.1 Non-Neural Baseline

Earlier approaches to morphological inflection primarily relied on explicit hand-crafted transformations to generate inflected word forms. In line with this tradition, the first baseline evaluated in this work adopts a simple edit-rule strategy.

The system is provided as part of the SIGMORPHON 2023 Shared Task (Goldman et al., 2023). It builds upon a baseline introduced in the SIGMORPHON 2020 edition (Vylomova et al., 2020) and is based on the model originally proposed by Cotterell et al. (2017).

Unlike neural sequence-to-sequence approaches, this model does not learn continuous representations. Instead, it derives explicit string edit rules directly from the training data. For each training triple consisting of a lemma, a set of morphological tags, and the corresponding inflected form, the system first aligns the two strings using Levenshtein distance. The aligned pair is then decomposed into a prefix part, a shared stem, and a suffix part. Based on this decomposition, both prefix-changing and suffix-changing transformation rules are extracted.

To illustrate this procedure, we reproduce a simplified alignment example adapted

from the official baseline documentation (Vylomova et al., 2020). For the German pair *schielen* $\rightarrow$ *geschielt*, one possible alignment is:

```
--schielen
geschielt_
```

which can be segmented into prefix (Pr), stem (St), and suffix (Su) regions:

```
Pr |   St   | Su
-- | schiele | n
ge | schielt | _
```

From this decomposition, the system extracts both prefix rules (e.g., $\epsilon \rightarrow ge$) and suffix rules describing the stem-final changes.

Each training example may therefore yield multiple candidate rules of varying specificity. During inference, the system first selects the rule whose left-hand side matches the longest portion of the lemma (suffixes for suffix rules or prefixes for prefix rules). If several rules of equal length apply, the most frequent one observed in the training data is chosen.

Table 2.1 illustrates representative examples of such transformations. For clarity, the examples focus on suffix edits, which are more common in English, although the same procedure is applied analogously to prefixes. By aggregating such examples across the training data, the model learns which modifications are most likely for a given morphological specification.

Importantly, each training pair contributes multiple candidate rules rather than a single transformation. For example, from the pair *walk* $\rightarrow$ *walked*, the alignment procedure yields a hierarchy of suffix edits with increasing specificity, including $\epsilon \rightarrow ed$, $k \rightarrow ked$, $lk \rightarrow lked$, $alk \rightarrow alked$, and *walk* $\rightarrow$ *walked*. This strategy encourages more specific transformations while still allowing general rules to apply when appropriate.

While this allows the model to efficiently capture regular patterns, it can lead to overgeneralization when training data is sparse. For instance, if the only observed pair were *speak* $\rightarrow$ *spoke*, the model could learn the rule *eak* $\rightarrow$ *oke*. Applying this rule to an unseen lemma such as *leak* would incorrectly yield *loke*, instead of *leaked*. This example illustrates how purely surface-based edits may fail to generalize appropriately.

Despite these limitations, the approach provides a simple, transparent, and computationally efficient baseline. Its explicit rule extraction makes the predictions easily interpretable and offers a useful reference point for comparing the more complex neural models introduced in the following sections.

Lemma	MSD	Inflected form	Example candidate rule
walk	V;PST;3;SG	walked	$\epsilon \rightarrow \text{ed}$
talk	V;PST;3;SG	talked	$\text{lk} \rightarrow \text{lked}$
play	V;PST;3;SG	played	$\text{ay} \rightarrow \text{ayed}$
jump	V;PST;3;SG	jumped	$\text{mp} \rightarrow \text{mped}$
call	V;PST;3;SG	called	$\text{all} \rightarrow \text{alled}$
love	V;PST;3;SG	loved	$\text{ve} \rightarrow \text{ved}$
move	V;PST;3;SG	moved	$\text{ove} \rightarrow \text{oved}$
try	V;PST;3;SG	tried	$\text{y} \rightarrow \text{ied}$
stop	V;PST;3;SG	stopped	$\text{p} \rightarrow \text{pped}$
fix	V;PRS;3;SG	fixes	$\epsilon \rightarrow \text{es}$

Table 2.1: Illustrative examples of edit rules extracted by the non-neural baseline. The table focuses on suffix rules for readability, although both prefix and suffix transformations are learned.

2.2.2 Neural Baseline

As a neural reference system, we use the official neural baseline provided as part of the SIGMORPHON 2023 Shared Task. This system corresponds to the character-level Transformer model proposed by Wu et al. (2021), which has been adopted as a strong baseline for morphological inflection.

The model follows a standard encoder-decoder Transformer architecture (Vaswani et al., 2017), operating directly on character sequences. To better suit low-resource transduction tasks, a compact configuration is used, consisting of four encoder layers and four decoder layers with multi-head self-attention. Each layer employs four attention heads, a model dimension of $d_{\text{model}} = 256$, and a feed-forward dimension of $d_{\text{FF}} = 1024$. Decoding is performed autoregressively. For feature-guided tasks such as morphological inflection, morphological tags are encoded as special feature tokens that are concatenated with the input character sequence and processed jointly by the encoder. Wu et al. further propose a feature-invariant positional encoding scheme to avoid imposing an arbitrary order over the feature set.

2.3 ByT5

In addition to character-level and token-based models, we consider ByT5 (Xue et al., 2022), a token-free variant of the T5 architecture. ByT5 is an encoder-decoder Transformer model that operates directly on UTF-8 byte sequences instead of subword

tokens. Unlike conventional Transformer models that rely on learned vocabularies (e.g., SentencePiece), ByT5 uses a fixed vocabulary of 256 byte embeddings corresponding to all possible byte values, thereby eliminating the need for tokenization.

Architecturally, ByT5 follows the T5 framework (Raffel et al., 2020) and its multilingual variant mT5 (Xue et al., 2021). To better accommodate longer byte sequences, the encoder and decoder depths are decoupled, employing a substantially heavier encoder stack with a 3:1 encoder-decoder ratio, in contrast to the balanced configuration used in T5 and mT5.

Because ByT5 operates directly on raw byte sequences, it has been hypothesized to be particularly suitable for tasks that are sensitive to orthographic or phonological structure, including morphological inflection. This property makes it especially relevant for the evaluation conducted in this thesis.

In our experiments, ByT5 is fine-tuned on the target morphological task. The fine-tuning configuration is described in Section 4.

2.4 Large Language Models

Large language models (LLMs) are Transformer-based neural networks pretrained on large-scale text corpora using self-supervised objectives. Owing to their scale and broad pretraining, such models exhibit strong generalization capabilities and can perform a wide range of sequence-to-sequence tasks with little or no task-specific training. Since both morphological inflection and morphological analysis can be formulated as text generation problems, LLMs provide a natural evaluation framework without dedicated fine-tuning.

In this work, we evaluate two open-weight instruction-tuned LLMs: Qwen3-8B (Yang et al., 2025) and Llama-3.1-8B-Instruct (Grattafiori et al., 2024). Both models are decoder-only Transformer architectures with approximately 8 billion parameters and are designed for general-purpose text generation. The 8B parameter scale represents a compromise between strong performance and practical computational requirements.

Qwen3-8B, developed by Alibaba, is part of the Qwen model family and is trained on large-scale multilingual data to provide strong general-purpose language understanding and generation capabilities. Llama-3.1-8B-Instruct, developed by Meta AI, belongs to the LLaMA series and incorporates improved pretraining data and alignment procedures relative to earlier versions. Both models are instruction-tuned to follow natural language prompts, making them suitable for prompt-based evaluation settings. In this thesis, they are used without additional fine-tuning and evaluated in a one-shot prompting setup.

3 Datasets and Languages

3.1 Datasets

To address the tasks of morphological inflection and morphological analysis and to train the proposed models, labeled data is required. Each instance consists of a lemma, a set of morphological tags describing the target grammatical features, and the corresponding inflected form. For this purpose, this work relies on two widely used resources: UniMorph (Batsuren et al., 2022) and Universal Dependencies (UD) (Nivre et al., 2020).

3.1.1 UniMorph

UniMorph is a multilingual project dedicated to providing large-scale morphological resources in the form of inflection tables and structured morphological annotations across a wide range of languages. At the time of writing, the dataset covers up to 182 languages, including both high-resource and low-resource as well as endangered languages.

For each language, UniMorph provides a dataset consisting of individual instances that pair a lemma with a bundle of morphological features and the corresponding inflected form. Concretely, the data is distributed as tab-separated lines, where each row represents a single lemma, tags, form triple. Multiple rows typically share the same lemma and correspond to different morphological feature combinations, effectively encoding the full inflectional paradigm in a flat, line-based format, as illustrated in Table 3.1.

Although the most recent UniMorph publication describes a transition from the previously used flat annotation scheme to a hierarchical structure for morphological tags (e.g., `V;PST;NOM(3,SG)`), the datasets currently distributed by the project remain annotated in the original flat format. Consequently, all data used in this thesis follows this flat annotation style.

UniMorph data served as the main resource throughout the experimental pipeline. As in the original SIGMORPHON 2023 Shared Task setup, it formed the basis for the reimplementation of the shared task baselines.

(a) UniMorph instance			(b) Tag interpretation	
Lemma	Tags	Inflected form	Tag	Meaning
walk	V;PST;3;SG	walked	V	Verb
			PST	Past tense
			3	Third person
			SG	Singular

Table 3.1: Example UniMorph instance in the tab-separated format used in this work (left), together with the interpretation of the corresponding morphological tags (right).

Furthermore, the dataset was used to fine-tune the ByT5 model for morphological inflection and, by inverting the source–target mapping, also for morphological analysis. It additionally constituted the core training data for the bidirectional model, while certain experimental settings were further supplemented with data derived from Universal Dependencies.

3.1.2 Universal Dependencies

The UD resource provides consistent morphosyntactic annotation across languages, offering dependency treebanks with standardized part-of-speech tags, morphological features, and syntactic relations for sentences. At the time of writing, UD covers over 150 languages through more than 200 treebanks, spanning both high-resource and low-resource settings.

The data is distributed in the CoNLL-U format, where each sentence is represented as a sequence of token-level annotations. In addition to syntactic dependency information, each token is associated with a lemma and a set of morphological features encoded as attribute–value pairs (e.g., Tense=Past|Number=Sing|Person=3). Compared to UniMorph’s tab-separated triple format, UD therefore provides richer contextual and sentence-level information alongside morphological annotations. An example taken from the Portuguese PetroGold treebank is shown in Figure 3.1.

In this work, UD data serves as a source of contextual information for fine-tuning the bidirectional ByT5 model and for conducting experiments with large language models.

Since UD often provides multiple treebanks for a given language, it was necessary to select a single resource for each case. To guide this choice, we relied on the ranking provided by the UD project, which evaluates treebanks based on their size, annotation quality, and linguistic diversity. Whenever multiple treebanks were available for a language, the highest-ranked treebank according to this score was selected.

```
# text = Estas podem representar falhas profundas.
# sent_id = 247-20140910-MONOGRAFIA_0-42
1 Estas este PRON _ Gender=Fem|Number=Plur|PronType=Dem 2 nsubj _ _
2 podem poder VERB _ Mood=Ind|Number=Plur|Person=3|Tense=Pres|VerbForm=Fin 0 root _ _
3 representar representar VERB _ VerbForm=Inf 2 xcomp _ _
4 falhas falha NOUN _ Gender=Fem|Number=Plur 3 obj _ _
5 profundas profundo ADJ _ Gender=Fem|Number=Plur 4 amod _ SpaceAfter=No
6 . . PUNCT _ _ 2 punct _ _
```

Figure 3.1: Example sentence in CoNLL-U format from the Portuguese UD PetroGold treebank (“These may represent deep faults.”).

3.1.3 Data Preparation

Since the two data sources use different annotation schemes and file formats, several preprocessing and conversion steps were required to obtain a unified representation suitable for all experiments.

UniMorph For UniMorph, the data was randomly sampled from the original datasets with a fixed seed, giving a preference to verbs, and then split into training, development, and test sets. This preference reflects that verbs typically exhibit the richest inflectional paradigms and constitute the primary focus of the morphological inflection task. These partitions consist of 10,000 training instances, 1,000 development instances, and 1,000 test instances, as in the Shared Task setup.

To prevent data leakage across splits, sampling and partitioning is performed at the lemma level, i.e., all inflected forms associated with a given lemma are always assigned to the same partition (Goldman et al., 2022). Verbal instances are identified using the UniMorph tag string, where entries whose feature bundle begins with *V* (including prefixed variants such as participles) are treated as verbs. If a split exceeds the target size, instances are subsampled within the split. Finally, all partitions are written in a unified tab-separated format (*lemma, tags, form*) to ensure consistency with the downstream baselines and model implementations.

While Table 3.3 reports the average number of inflected forms per lemma across the complete UniMorph datasets, this value varies substantially across parts of speech. In particular, Georgian exhibits a much richer verbal paradigm than suggested by the overall average. When considering verbs only, the average number of forms per lemma increases from approximately 24 to 178 forms. Because the data preparation procedure prioritizes verbal instances, this discrepancy has a noticeable impact on the effective data distribution and on the resulting train–development–test splits for Georgian.

To maintain comparable dataset sizes across languages, the preprocessing script allows specifying a maximum number of forms retained per lemma. For Georgian,

a language-specific limit of 15 forms was applied in order to prevent individual paradigms from dominating the sampled data while preserving broad lemma coverage.

Universal Dependencies For most UD treebanks, predefined training, development, and test splits are provided. However, for Amharic only a test set is available. In this case, this data was randomly partitioned into training, development, and test sets following an 8/1/1 proportion, as no official split specification is provided by UD and no common ratio can be derived from the analyzed languages.

To integrate UD data into the same downstream pipeline as UniMorph, all instances are converted into a unified tab-separated format that closely follows the UniMorph specification. Each instance consists of a lemma, a set of morphological tags, and the corresponding inflected form, separated by tab characters; for contextual settings, an additional fourth column containing the sentence context is included.

To incorporate UD data into this format, the morphological feature annotations provided in the CoNLL-U files were first mapped to their UniMorph equivalents. For this purpose, we used the official UniMorph conversion tool, which translates UD feature sets into the UniMorph schema using language-specific translators (McCarthy et al., 2018). Since Georgian and Amharic are not supported by the provided translators, the experiments requiring UD data for them were done with the original UD annotation style.

Figure 3.2 illustrates the output of the conversion tool when applied to the sentence shown in Figure 3.1. The UniMorph-compatible tags replace the original UD feature representations while preserving the original tokenization and syntactic structure.

```
# text = Estas podem representar falhas profundas.
# sent_id = 247-20140910-MONOGRAFIA_0-42
1 Estas este PRON _ PRO;PL;FEM 2 nsubj _ _
2 podem poder VERB _ PL;PRS;V;IND;3 0 root _ _
3 representar representar VERB _ V;NFIN 2 xcomp _ _
4 falhas falha NOUN _ PL;FEM;N 3 obj _ _
5 profundas profundo ADJ _ PL;FEM;ADJ 4 amod _ SpaceAfter=No
6 . . PUNCT _ _ 2 punct _ _
```

Figure 3.2: Example output from official conversion tool.

Since the UniMorph data predominantly contains verbs, UD data was filtered accordingly to include only verbal tokens. Concretely, for each sentence, the first verb was selected and used to construct a single instance.

After conversion, the lemma, the inflected form, the mapped UniMorph tags, and the surrounding sentence context were extracted to construct the final tab-separated instances used in the downstream experiments. The resulting unified format is illustrated

in Table 3.2.

Lemma	Tags	Inflected form	Context
poder	V;IND;PRS;PL;3	podem	Estas podem representar falhas profundas.

Table 3.2: Example instance of the unified tab-separated format used in this work.

All preprocessing, conversion, and extraction scripts are included in the accompanying code repository to ensure full reproducibility.

3.2 Languages

The following tables summarize the languages considered in the experiments. Table 3.3 presents general typological and linguistic information, together with the linguistic diversity score proposed by Joshi et al. (2020) as an external measure of language resource availability. In addition, the total number of UniMorph instances is reported to reflect the overall amount of available training data. The average number of inflected forms per lemma is included as a coarse estimate of morphological richness.

Table 3.4 provides statistics for the Universal Dependencies resources. Since UD treebanks are not available for Amuzgo and Lower Sorbian, these languages are excluded from the contextual and large language model experiments. For the remaining languages, the selected treebank and the sizes of the training, development, and test splits are listed; for Amharic, these correspond to the splits obtained by randomly partitioning the original UD test-only data into train/dev/test.

Language	ISO 639-3	Family	Type	Ling. score	UM size	Avg. forms
Amharic	amh	Semitic	Templatic	2	46224	18.78
Amuzgo	azg	Oto-Manguean	Polysynthetic	0	12204	36.76
Ancient Greek	grc	IE/Hellenic	Fusional	3	41593	18.10
Lower Sorbian	dsb	IE/Slavic	Fusional	1	20121	20.22
English	eng	IE/Germanic	Analytic	5	115523	1.63
Italian	ita	IE/Romance	Fusional	4	509574	50.91
Georgian	kat	Kartvelian	Agglutinative	3	74412	23.90
Portuguese	por	IE/Romance	Fusional	4	303996	75.96

Table 3.3: Typological overview and UniMorph statistics for the selected languages. Language identifiers follow ISO 639-3 and are used throughout the Results tables. For readability, Indo-European language families are prefixed with *IE* (Indo-European). *Ling. score* (0–5; higher means better represented in NLP resources and literature) provides a coarse external indicator of language resource availability. *UM size* denotes the number of UniMorph instances (lemma–tag bundle–inflected form triples) available for each language.

Language	ISO 639-3	Treebank	Train	Dev	Test
Amharic	amh	ATT	811	102	99
Ancient Greek	grc	PTNK	692	424	404
English	eng	GUM	7692	1220	1171
Italian	ita	ISDT	10728	475	404
Georgian	kat	GNC	798	120	741
Portuguese	por	PetroGold	5871	568	894

Table 3.4: Universal Dependencies treebanks and dataset sizes for the languages with available UD resources.

4 Methodology

All experiments in this thesis are executed through a unified runner that separates implementation from configuration. Model- and language-specific settings are defined in YAML configuration files (including data paths, hyperparameters, and prompt templates), while a single entry point orchestrates training and inference across languages. To ensure consistent reporting across model families and tasks, each run writes a language-specific JSON file containing the evaluation metrics and relevant metadata, and additionally appends a summary row to a central CSV file aggregating all results. The full set of scripts, configurations, and evaluation utilities used to produce the reported results is made available in the accompanying repository to support complete reproducibility.

4.1 Morphological Inflection

This section describes how the different systems are trained and evaluated for morphological inflection. For each model, we summarize the concrete experimental procedure (data interface, training and inference setup, and implementation details relevant for reproducibility). Unless stated otherwise, all models operate on the unified UniMorph-based train–development–test splits described in Chapter 3.

4.1.1 Non-Neural Baseline

As a non-neural reference system, we use the official SIGMORPHON shared-task implementation (Goldman et al., 2023) described in Section 2.2.1. The baseline code is used as provided¹, with a minor command-line extension that allows restricting execution to a single language (via a `-lang` argument) for convenience and to simplify batched experiment runs.

4.1.2 Neural Baseline

As a neural reference system, we use the official SIGMORPHON 2023 neural transducer baseline (Goldman et al., 2023) described in Section 2.2.2. Training is performed by

¹<https://github.com/sigmorphon/2023InflectionST/blob/main/part1/baselines/nonneural.py>

invoking the shared-task example script², which calls the baseline training code with a fixed configuration and takes the language code as an argument.

For each language, the model is trained on the corresponding training split, monitored on the development split, and evaluated on the test split. The shared-task training code produces a decoded prediction file in TSV format for the test set, which we use as model outputs when computing and reporting accuracy.

Table 4.1 summarizes the hyperparameters used in our experiments as specified by the shared-task script.

Hyperparameter	Value
Architecture	feature-invariant Transformer (tagtransformer)
Encoder / decoder layers	4 / 4
Embedding dimension	256
Hidden size (d_{FF} / internal)	1024
Attention heads	4
Dropout	0.3
Batch size	400
Learning rate	0.001
Scheduler	warmupinvsqr (warmup: 4000 steps)
Training budget	20000 update steps (max_steps)
Label smoothing	0.1
Adam β_2	0.98
Gradient clipping	max norm 1
Decoding	greedy, max decode length 32
Checkpoint selection	best development accuracy (-bestacc)

Table 4.1: Hyperparameters of the SIGMORPHON 2023 neural baseline as used in our experiments (taken from `example/sigmorphon2023-shared-tasks/task0-trm.sh`).

4.1.3 ByT5

For morphological inflection, we fine-tune a pretrained ByT5 model using the Hugging-Face Transformers framework. Concretely, we load the `google/byt5-small` checkpoint via `AutoTokenizer` and `AutoModelForSeq2SeqLM` and train using PyTorch with the `Seq2SeqTrainer` API. Training is performed on GPU.

²<https://github.com/omagolda/neural-transducer/tree/master/example/sigmorphon2023-shared-tasks>

Each training instance is represented as a sequence-to-sequence example. The input string is constructed from the lemma and its UniMorph feature bundle using a fixed prompt template (Figure 4.1), and the target string is the corresponding inflected form. Inputs are processed with truncation to a maximum length of 128.

Generate the inflected form for: {lemma} {features}

Figure 4.1: Prompt template used to construct the ByT5 input sequence for morphological inflection.

Table 4.2 summarizes the training and decoding hyperparameters used in our experiments.

Hyperparameter	Value
Pretrained checkpoint	google/byt5-small
Epochs	3
Learning rate	1×10^{-4}
Scheduler	linear (warmup: 100 steps)
Weight decay	0.01
Batch size (train / eval, per device)	8 / 8
Mixed precision	fp16 disabled
Seed	42
Evaluation / saving	every 50 / 100 steps (keep last 1 checkpoint)
Max input length (encoding)	128
Max target length (encoding)	128
Decoding	greedy
Max generation length (eval/test)	128
Prediction batch size	16

Table 4.2: ByT5 fine-tuning hyperparameters used for the morphological inflection experiments.

4.1.4 Evaluation

All models are evaluated on held-out test instances by comparing each generated output to the gold label provided in the corresponding dataset test split. For the morphological inflection task, the primary metric is exact-match accuracy on the inflected form, i.e., the fraction of test instances for which the predicted inflected form exactly matches the

gold form. String comparisons are performed after applying Unicode normalization (NFC), lowercasing, and whitespace trimming.

In addition to accuracy, we report the mean Levenshtein distance between the predicted and gold inflected forms (computed on the same normalized strings) as a complementary measure that captures the magnitude of character-level deviations when the prediction is not exactly correct.

4.2 Morphological Analysis

This section describes the experimental setup for morphological analysis.

4.2.1 ByT5

For the analysis ByT5 model, we fine-tune ByT5 in the inverse direction on the UniMorph-based dataset. The only change relative to inflection is the input–output mapping and the corresponding prompt template: the model receives an inflected form as input and generates a single target string of the form “lemma FEATURES”, where the lemma is followed by the semicolon-separated UniMorph tag bundle (separated by whitespace). We use the inverse prompt template shown in Figure 4.3.

Encoding and optimization hyperparameters are identical to the inflection ByT5 setup (Table 4.2).

4.2.2 Large Language Models

We evaluate large language models in an inference-only setting via the OpenRouter chat-completions API on the UD-derived unified dataset with context. Each instance is processed independently, i.e., every prediction is generated through a separate API request without retaining conversational history between examples. To reduce stochasticity and improve reproducibility, we fix the decoding configuration across all experiments by setting the temperature to 0.0 and by using a constant random seed.

We employ a one-shot prompting strategy. Each prompt consists of (i) a short instruction describing the task, (ii) a single labeled example drawn from the first data point of the development split, and (iii) the target instance for which a prediction is required. The same template and demonstration example are included in every request to ensure consistent behavior across instances. An example of the prompt template is shown in Figure 4.2.

```

Based on this example:
Input: {example_form}
Context: {example_context}
Prediction: {example_lemma}\t{example_tags}

Generate the lemma and morphological tags for the following inflected
verb and context. IMPORTANT: Provide ONLY the answer in the exact format
'lemma\ttags' with NO explanations, NO commentary, NO additional text:
{form}\t{context}

Answer (lemma and tags only, no explanation):

```

Figure 4.2: Prompt structure used for the one-shot LLM baseline. The target instance consists of an inflected form together with its sentence context.

4.2.3 Evaluation

For morphological analysis, the target output is a single string of the form “lemma FEATURES”. The evaluation splits the prediction and the gold label at the first whitespace into a lemma and an morphosyntactic description (MSD). The MSD component is interpreted as a case-insensitive set of semicolon-separated tags.

We report (i) lemma exact-match accuracy, and (ii) the mean Levenshtein distance between the predicted and gold lemmas (computed on normalized strings) to quantify character-level deviations. For MSD, we compute (iii) exact set-match accuracy (prediction set equals gold set) and (iv) micro-averaged precision, recall, and F_1 over tags, where true positives, false positives, and false negatives are accumulated across all test instances before computing the final scores.

For Georgian and Amharic in the UD-based context experiments, the available UD feature bundles cannot be converted to the UniMorph format due to missing language-specific translators. Since this yields feature inventories that are not directly comparable to the UniMorph-style MSD tags used for other languages, we do not report the MSD-based scores for these two languages, and instead report only the lemma metrics.

4.3 Bidirectional Inflection and Analysis

In addition to the task-specific ByT5 models, we fine-tune a single bidirectional model that jointly learns morphological inflection and morphological analysis within a unified sequence-to-sequence framework. Concretely, for each language we shuffle the Uni-

Morph training split with a fixed random seed and partition it into two equally sized subsets. One subset is converted into forward (inflection) examples (lemma + tags $\rightarrow$ inflected form) using the inflection prompt template (Figure 4.1), while the other subset is converted into inverse (analysis) examples (inflected form $\rightarrow$ lemma + tags) using a separate prompt template (Figure 4.3). The combined dataset is then used to fine-tune a single model.

The same procedure is applied to the development split to form a mixed-direction validation set. At test time, we evaluate the bidirectional model in the forward direction for morphological inflection and in the inverse direction for morphological analysis.

```
Generate the lemma and morphological tags for the following inflected
verb: {form}
```

Figure 4.3: Prompt template used to construct the ByT5 input sequence for the inverse mapping (morphological analysis) in the bidirectional setting.

Table 4.3 summarizes the hyperparameters used for bidirectional fine-tuning and decoding.

Hyperparameter	Value
Pretrained checkpoint	google/byt5-small
Training directions	50% forward + 50% inverse (random split)
Epochs	3
Learning rate	1×10^{-4}
Scheduler	linear (warmup: 100 steps)
Weight decay	0.01
Batch size (train / eval, per device)	8 / 8
Mixed precision	fp16 disabled
Seed	42
Evaluation / saving	every 50 / 100 steps (keep last 1 checkpoint)
Max input length (encoding)	128
Max target length (encoding)	128
Decoding	greedy
Max generation length (eval/test)	128
Prediction batch size	16

Table 4.3: ByT5 bidirectional fine-tuning hyperparameters used in our experiments.

4.4 Context-Aware Morphological Analysis

To incorporate sentence context for morphological analysis, we start from the language-specific bidirectional ByT5 model trained in Section 4.3 and further fine-tune it on the UD-derived unified dataset that provides sentence context (one model per language). Table 4.4 summarizes the key differences introduced by this additional fine-tuning stage relative to the standard bidirectional setup. All remaining optimization and decoding hyperparameters are taken unchanged from Table 4.3.

Hyperparameter	Value
Pretrained checkpoint	results/byt5-bidir-{lang}/checkpoint-3750
Inference direction	inverse only
Maximum context length	512

Table 4.4: Key differences between the context-aware model and the standard bidirectional ByT5 setup (Table 4.3).

At inference time, we decode only in the inverse (analysis) direction.

For each instance, we include the sentence context from the UD-derived dataset (the sentence in which the target inflected verb occurs) directly in the model input via the prompt. The resulting template is shown in Figure 4.4.

```
Given the context:  '{context}'
Generate the lemma and morphological tags for the inflected verb:
{form}
```

Figure 4.4: Prompt template used for the context-aware ByT5 inverse model.

5 Results

5.1 Morphological Inflection

This section reports test-set performance on the UniMorph-based forward inflection task. Results are summarized as matrices across languages: accuracy in Table 5.1 and mean Levenshtein distance in Table 5.2.

Model	amh	azg	dsb	eng	grc	ita	kat	por
Non-neural	0.390	0.061	0.852	0.958	0.290	0.762	0.842	0.966
Neural	0.887	0.201	0.791	0.966	0.194	0.783	0.892	0.976
ByT5	0.822	0.265	0.839	0.971	0.416	0.958	0.851	0.950
ByT5 bi	0.608	0.244	0.783	0.976	0.348	0.898	0.829	0.911

Table 5.1: Forward inflection accuracy ($\uparrow$). Language columns use ISO 639-3 codes (Table 3.3).

Model	amh	azg	dsb	eng	grc	ita	kat	por
Non-neural	1.010	5.660	0.267	0.086	1.852	0.761	0.430	0.054
Neural	0.162	3.703	0.379	0.069	2.212	0.266	0.297	0.031
ByT5	0.237	3.413	0.287	0.067	1.713	0.091	0.480	0.072
ByT5 bi	0.606	3.564	0.375	0.050	2.020	0.243	0.634	0.149

Table 5.2: Forward inflection mean Levenshtein distance ($\downarrow$). Language columns use ISO 639-3 codes (Table 3.3).

5.2 Morphological Analysis

This section reports test-set performance on morphological analysis. As in the evaluation pipeline, results are reported for (i) predicting the lemma and (ii) predicting the

target MSD (interpreted as a set of tags). Tables 5.3–5.6 summarize lemma and MSD metrics as matrices across languages.

Model	amh	azg	dsb	eng	grc	ita	kat	por
ByT5 inv	0.762	0.344	0.613	0.970	0.597	0.945	0.756	0.948
ByT5 bi-inv	0.646	0.371	0.643	0.968	0.640	0.926	0.762	0.942
ByT5 ctx	0.899	–	–	0.969	0.144	0.955	0.061	0.745
LLM Llama	0.667	–	–	0.664	0.351	0.782	0.049	0.713
LLM Qwen	0.475	–	–	0.491	0.327	0.428	0.040	0.516

Table 5.3: Morphological analysis: Lemma accuracy ($\uparrow$). Language columns use ISO 639-3 codes (Table 3.3).

Model	amh	azg	dsb	eng	grc	ita	kat	por
ByT5 inv	0.287	2.618	0.597	0.044	0.737	0.099	0.462	0.076
ByT5 bi-inv	0.408	2.343	0.546	0.048	0.701	0.141	0.434	0.091
ByT5 ctx	0.131	–	–	0.072	4.030	0.149	3.188	0.596
LLM Llama	0.798	–	–	0.855	4.057	2.002	4.750	0.751
LLM Qwen	4.323	–	–	5.465	6.077	8.332	17.819	8.453

Table 5.4: Morphological analysis: Lemma mean Levenshtein ($\downarrow$). Language columns use ISO 639-3 codes (Table 3.3).

a

Model	amh	azg	dsb	eng	grc	ita	kat	por
ByT5 inv	0.832	0.778	0.462	0.813	0.734	0.789	0.727	0.661
ByT5 bi-inv	0.723	0.692	0.368	0.817	0.638	0.781	0.730	0.657
ByT5 ctx	0.778	–	–	0.957	0.136	0.968	0.057	0.718
LLM Llama	0.273	–	–	0.105	0.366	0.035	0.148	0.019
LLM Qwen	0.404	–	–	0.067	0.478	0.218	0.108	0.028

Table 5.5: Morphological analysis: MSD accuracy ($\uparrow$). Language columns use ISO 639-3 codes (Table 3.3).

Model	amh	azg	dsb	eng	grc	ita	kat	por
ByT5 inv	0.928	0.944	0.724	0.964	0.906	0.930	0.870	0.890
ByT5 bi-inv	0.884	0.890	0.679	0.966	0.849	0.925	0.849	0.885
ByT5 ctx	0.916	–	–	0.983	0.306	0.984	0.486	0.838
LLM Llama	0.622	–	–	0.375	0.817	0.177	0.737	0.458
LLM Qwen	0.652	–	–	0.407	0.818	0.247	0.436	0.415

Table 5.6: Morphological analysis: MSD F1 ($\uparrow$). Language columns use ISO 639-3 codes (Table 3.3).

6 Conclusion

Placeholder text.

Abbreviations

TUM Technical University of Munich

UD Universal Dependencies

MSD morphosyntactic description

List of Figures

1.1	Example drawing	1
1.2	Example plot	2
1.3	Example listing	2
2.1	Illustration of the morphological inflection task. Given a lemma and a set of grammatical features, the goal is to generate the corresponding inflected form.	3
2.2	Illustration of the morphological analysis task. Given an inflected form, the goal is to predict the corresponding lemma and set of grammatical features.	4
3.1	Example sentence in CoNLL-U format from the Portuguese UD Petro-Gold treebank ("These may represent deep faults.").	11
3.2	Example output from official conversion tool.	12
4.1	Prompt template used to construct the ByT5 input sequence for morphological inflection.	17
4.2	Prompt structure used for the one-shot LLM baseline. The target instance consists of an inflected form together with its sentence context.	19
4.3	Prompt template used to construct the ByT5 input sequence for the inverse mapping (morphological analysis) in the bidirectional setting.	20
4.4	Prompt template used for the context-aware ByT5 inverse model.	21

List of Tables

1.1	Example table	1
2.1	Illustrative examples of edit rules extracted by the non-neural baseline. The table focuses on suffix rules for readability, although both prefix and suffix transformations are learned.	7
3.1	Example UniMorph instance in the tab-separated format used in this work (left), together with the interpretation of the corresponding morphological tags (right).	10
3.2	Example instance of the unified tab-separated format used in this work.	13
3.3	Typological overview and UniMorph statistics for the selected languages. Language identifiers follow ISO 639-3 and are used throughout the Results tables. For readability, Indo-European language families are prefixed with <i>IE</i> (Indo-European). <i>Ling. score</i> (0–5; higher means better represented in NLP resources and literature) provides a coarse external indicator of language resource availability. <i>UM size</i> denotes the number of UniMorph instances (lemma–tag bundle–inflected form triples) available for each language.	14
3.4	Universal Dependencies treebanks and dataset sizes for the languages with available UD resources.	14
4.1	Hyperparameters of the SIGMORPHON 2023 neural baseline as used in our experiments (taken from <code>example/sigmorphon2023-shared-tasks/task0-trm.sh</code>). 16	
4.2	ByT5 fine-tuning hyperparameters used for the morphological inflection experiments.	17
4.3	ByT5 bidirectional fine-tuning hyperparameters used in our experiments.	20
4.4	Key differences between the context-aware model and the standard bidirectional ByT5 setup (Table 4.3).	21
5.1	Forward inflection accuracy (↑). Language columns use ISO 639-3 codes (Table 3.3).	22
5.2	Forward inflection mean Levenshtein distance (↓). Language columns use ISO 639-3 codes (Table 3.3).	22

List of Tables

5.3	Morphological analysis: Lemma accuracy ($\uparrow$). Language columns use ISO 639-3 codes (Table 3.3).	23
5.4	Morphological analysis: Lemma mean Levenshtein ($\downarrow$). Language columns use ISO 639-3 codes (Table 3.3).	23
5.5	Morphological analysis: MSD accuracy ($\uparrow$). Language columns use ISO 639-3 codes (Table 3.3).	24
5.6	Morphological analysis: MSD F1 ($\uparrow$). Language columns use ISO 639-3 codes (Table 3.3).	24

Bibliography

- Aharoni, R. and Y. Goldberg (July 2017). “Morphological Inflection Generation with Hard Monotonic Attention.” In: *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Ed. by R. Barzilay and M.-Y. Kan. Vancouver, Canada: Association for Computational Linguistics, pp. 2004–2015. doi: 10.18653/v1/P17-1183.
- Batsuren, K., O. Goldman, S. Khalifa, N. Habash, W. Kieraś, G. Bella, B. Leonard, G. Nicolai, K. Gorman, Y. G. Ate, M. Ryskina, S. Mielke, E. Budianskaya, C. El-Khaissi, T. Pimentel, M. Gasser, W. A. Lane, M. Raj, M. Coler, J. R. M. Samame, D. S. Camaiteri, E. Z. Rojas, D. López Francis, A. Oncevay, J. López Bautista, G. C. S. Villegas, L. T. Hennigen, A. Ek, D. Guriel, P. Dirix, J.-P. Bernardy, A. Scherbakov, A. Bayyr-ool, A. Anastasopoulos, R. Zariquiey, K. Sheifer, S. Ganieva, H. Cruz, R. Karahóga, S. Markantonatou, G. Pavlidis, M. Plugaryov, E. Klyachko, A. Salehi, C. Angulo, J. Baxi, A. Krizhanovsky, N. Krizhanovskaya, E. Salesky, C. Vania, S. Ivanova, J. White, R. H. Maudslay, J. Valvoda, R. Zmigrod, P. Czarnowska, I. Nikkarinen, A. Salchak, B. Bhatt, C. Straughn, Z. Liu, J. N. Washington, Y. Pinter, D. Ataman, M. Wolinski, T. Suhardijanto, A. Yablonskaya, N. Stoehr, H. Dolatian, Z. Nuriah, S. Ratan, F. M. Tyers, E. M. Ponti, G. Aiton, A. Arora, R. J. Hatcher, R. Kumar, J. Young, D. Rodionova, A. Yemelina, T. Andrushko, I. Marchenko, P. Mashkovtseva, A. Serova, E. Prud’hommeaux, M. Nepomniashchaya, F. Giunchiglia, E. Chodroff, M. Hulden, M. Silfverberg, A. D. McCarthy, D. Yarowsky, R. Cotterell, R. Tsarfaty, and E. Vylomova (June 2022). “UniMorph 4.0: Universal Morphology.” In: *Proceedings of the Thirteenth Language Resources and Evaluation Conference*. Ed. by N. Calzolari, F. Béchet, P. Blache, K. Choukri, C. Cieri, T. Declerck, S. Goggi, H. Isahara, B. Maegaard, J. Mariani, H. Mazo, J. Odijk, and S. Piperidis. Marseille, France: European Language Resources Association, pp. 840–855.
- Beesley, K. R. and L. Karttunen (2003). *Finite-State Morphology*. Stanford, CA: CSLI Publications. ISBN: 978-1-57586-434-1.
- Botha, J. and P. Blunsom (22–24 Jun 2014). “Compositional Morphology for Word Representations and Language Modelling.” In: *Proceedings of the 31st International Conference on Machine Learning*. Ed. by E. P. Xing and T. Jebara. Vol. 32. Proceedings of Machine Learning Research 2. Beijing, China: PMLR, pp. 1899–1907.

- Cotterell, R. and G. Heigold (Sept. 2017). “Cross-lingual Character-Level Neural Morphological Tagging.” In: *Proceedings of the 2017 Conference on Empirical Methods in Natural Language Processing*. Ed. by M. Palmer, R. Hwa, and S. Riedel. Copenhagen, Denmark: Association for Computational Linguistics, pp. 748–759. doi: 10.18653/v1/D17-1078.
- Cotterell, R., C. Kirov, J. Sylak-Glassman, G. Walther, E. Vylomova, A. D. McCarthy, K. Kann, S. J. Mielke, G. Nicolai, M. Silfverberg, D. Yarowsky, J. Eisner, and M. Hulden (Oct. 2018). “The CoNLL–SIGMORPHON 2018 Shared Task: Universal Morphological Reinflection.” In: *Proceedings of the CoNLL–SIGMORPHON 2018 Shared Task: Universal Morphological Reinflection*. Ed. by M. Hulden and R. Cotterell. Brussels: Association for Computational Linguistics, pp. 1–27. doi: 10.18653/v1/K18-3001.
- Cotterell, R., C. Kirov, J. Sylak-Glassman, G. Walther, E. Vylomova, P. Xia, M. Faruqui, S. Kübler, D. Yarowsky, J. Eisner, and M. Hulden (Aug. 2017). “CoNLL–SIGMORPHON 2017 Shared Task: Universal Morphological Reinflection in 52 Languages.” In: *Proceedings of the CoNLL SIGMORPHON 2017 Shared Task: Universal Morphological Reinflection*. Ed. by M. Hulden. Vancouver: Association for Computational Linguistics, pp. 1–30. doi: 10.18653/v1/K17-2001.
- Faruqui, M., Y. Tsvetkov, G. Neubig, and C. Dyer (June 2016). “Morphological Inflection Generation Using Character Sequence to Sequence Learning.” In: *Proceedings of the 2016 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*. Ed. by K. Knight, A. Nenkova, and O. Rambow. San Diego, California: Association for Computational Linguistics, pp. 634–643. doi: 10.18653/v1/N16-1077.
- Goldman, O., K. Batsuren, S. Khalifa, A. Arora, G. Nicolai, R. Tsarfaty, and E. Vylomova (July 2023). “SIGMORPHON–UniMorph 2023 Shared Task 0: Typologically Diverse Morphological Inflection.” In: *Proceedings of the 20th SIGMORPHON workshop on Computational Research in Phonetics, Phonology, and Morphology*. Ed. by G. Nicolai, E. Chodroff, F. Mailhot, and Ç. Çöltekin. Toronto, Canada: Association for Computational Linguistics, pp. 117–125. doi: 10.18653/v1/2023.sigmorphon-1.13.
- Goldman, O., D. Guriel, and R. Tsarfaty (May 2022). “(Un)solving Morphological Inflection: Lemma Overlap Artificially Inflates Models’ Performance.” In: *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*. Ed. by S. Muresan, P. Nakov, and A. Villavicencio. Dublin, Ireland: Association for Computational Linguistics, pp. 864–870. doi: 10.18653/v1/2022.acl-short.96.
- Grattafiori, A. et al. (2024). *The Llama 3 Herd of Models*. arXiv: 2407.21783 [cs.AI].
- Joshi, P., S. Santy, A. Budhiraja, K. Bali, and M. Choudhury (2020). “The State and Fate of Linguistic Diversity and Inclusion in the NLP World.” In: *Proceedings of the 58th Annual*

- Meeting of the Association for Computational Linguistics*. DOI: 10.18653/v1/2020.ac1-main.560.
- Kann, K. and H. Schütze (Aug. 2016). "Single-Model Encoder-Decoder with Explicit Morphological Representation for Reinflection." In: *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*. Ed. by K. Erk and N. A. Smith. Berlin, Germany: Association for Computational Linguistics, pp. 555–560. DOI: 10.18653/v1/P16-2090.
- Kaplan, R. M. and M. Kay (1994). "Regular Models of Phonological Rule Systems." In: *Computational Linguistics* 20.3. Ed. by J. Hirschberg, pp. 331–378.
- Koskenniemi, K. (1983). *Two-Level Morphology: A General Computational Model for Word-Form Recognition and Production*. English. Publications 11. Finland: University of Helsinki. Department of General Linguistics. ISBN: 951-45-3201-5.
- Lamport, L. (1994). *LaTeX : A Documentation Preparation System User's Guide and Reference Manual*. Addison-Wesley Professional.
- Malaviya, C., S. Wu, and R. Cotterell (June 2019). "A Simple Joint Model for Improved Contextual Neural Lemmatization." In: *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*. Ed. by J. Burstein, C. Doran, and T. Solorio. Minneapolis, Minnesota: Association for Computational Linguistics, pp. 1517–1528. DOI: 10.18653/v1/N19-1155.
- McCarthy, A. D., M. Silfverberg, R. Cotterell, M. Hulden, and D. Yarowsky (Nov. 2018). "Marrying Universal Dependencies and Universal Morphology." In: *Proceedings of the Second Workshop on Universal Dependencies (UDW 2018)*. Ed. by M.-C. de Marneffe, T. Lynn, and S. Schuster. Brussels, Belgium: Association for Computational Linguistics, pp. 91–101. DOI: 10.18653/v1/W18-6011.
- Nivre, J., M.-C. de Marneffe, F. Ginter, J. Hajič, C. D. Manning, S. Pyysalo, S. Schuster, F. Tyers, and D. Zeman (May 2020). "Universal Dependencies v2: An Evergrowing Multilingual Treebank Collection." eng. In: *Proceedings of the Twelfth Language Resources and Evaluation Conference*. Ed. by N. Calzolari, F. Béchet, P. Blache, K. Choukri, C. Cieri, T. Declerck, S. Goggi, H. Isahara, B. Maegaard, J. Mariani, H. Mazo, A. Moreno, J. Odijk, and S. Piperidis. Marseille, France: European Language Resources Association, pp. 4034–4043. ISBN: 979-10-95546-34-4.
- Raffel, C., N. Shazeer, A. Roberts, K. Lee, S. Narang, M. Matena, Y. Zhou, W. Li, and P. J. Liu (2020). "Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer." In: *Journal of Machine Learning Research* 21.140, pp. 1–67.
- Sennrich, R., B. Haddow, and A. Birch (Aug. 2016). "Neural Machine Translation of Rare Words with Subword Units." In: *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Ed. by K. Erk and N. A.

- Smith. Berlin, Germany: Association for Computational Linguistics, pp. 1715–1725. DOI: 10.18653/v1/P16-1162.
- Vaswani, A., N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin (2017). “Attention is All you Need.” In: *Advances in Neural Information Processing Systems*. Ed. by I. Guyon, U. V. Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett. Vol. 30. Curran Associates, Inc.
- Vylomova, E., J. White, E. Salesky, S. J. Mielke, S. Wu, E. M. Ponti, R. H. Maudslay, R. Zmigrod, J. Valvoda, S. Toldova, F. Tyers, E. Klyachko, I. Yegorov, N. Krizhanovsky, P. Czarnowska, I. Nikkarinen, A. Krizhanovsky, T. Pimentel, L. Torroba Hennigen, C. Kirov, G. Nicolai, A. Williams, A. Anastasopoulos, H. Cruz, E. Chodroff, R. Cotterell, M. Silfverberg, and M. Hulden (July 2020). “SIGMORPHON 2020 Shared Task 0: Typologically Diverse Morphological Inflection.” In: *Proceedings of the 17th SIGMORPHON Workshop on Computational Research in Phonetics, Phonology, and Morphology*. Ed. by G. Nicolai, K. Gorman, and R. Cotterell. Online: Association for Computational Linguistics, pp. 1–39. DOI: 10.18653/v1/2020.sigmorphon-1.1.
- Wu, S., R. Cotterell, and M. Hulden (Apr. 2021). “Applying the Transformer to Character-level Transduction.” In: *Proceedings of the 16th Conference of the European Chapter of the Association for Computational Linguistics: Main Volume*. Ed. by P. Merlo, J. Tiedemann, and R. Tsarfaty. Online: Association for Computational Linguistics, pp. 1901–1907. DOI: 10.18653/v1/2021.eacl-main.163.
- Xue, L., A. Barua, N. Constant, R. Al-Rfou, S. Narang, M. Kale, A. Roberts, and C. Raffel (2022). “ByT5: Towards a Token-Free Future with Pre-trained Byte-to-Byte Models.” In: *Transactions of the Association for Computational Linguistics* 10. Ed. by B. Roark and A. Nenkova, pp. 291–306. DOI: 10.1162/tac1_a_00461.
- Xue, L., N. Constant, A. Roberts, M. Kale, R. Al-Rfou, A. Siddhant, A. Barua, and C. Raffel (June 2021). “mT5: A Massively Multilingual Pre-trained Text-to-Text Transformer.” In: *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*. Online: Association for Computational Linguistics, pp. 483–498. DOI: 10.18653/v1/2021.naacl-main.41.
- Yang, A., A. Li, B. Yang, B. Zhang, B. Hui, B. Zheng, B. Yu, C. Gao, C. Huang, C. Lv, C. Zheng, D. Liu, F. Zhou, F. Huang, F. Hu, H. Ge, H. Wei, H. Lin, J. Tang, J. Yang, J. Tu, J. Zhang, J. Yang, J. Yang, J. Zhou, J. Zhou, J. Lin, K. Dang, K. Bao, K. Yang, L. Yu, L. Deng, M. Li, M. Xue, M. Li, P. Zhang, P. Wang, Q. Zhu, R. Men, R. Gao, S. Liu, S. Luo, T. Li, T. Tang, W. Yin, X. Ren, X. Wang, X. Zhang, X. Ren, Y. Fan, Y. Su, Y. Zhang, Y. Zhang, Y. Wan, Y. Liu, Z. Wang, Z. Cui, Z. Zhang, Z. Zhou, and Z. Qiu (2025). *Qwen3 Technical Report*. arXiv: 2505.09388 [cs.CL].